Recurrent Neural Networks

Sequence Modeling, LSTM, Bidirectional LSTM & Sentiment Analysis

1 Sequence Modeling Using RNN

1.1 Why Sequence Modeling is Different

Standard feedforward networks — MLPs and CNNs — assume that every input is independent of every other input. Given an input vector $\mathbf{x}$, the network produces an output $\hat{y}$ with no memory of what it processed before. This assumption is valid for tasks like image classification, where one image truly has nothing to do with the next. But it breaks down immediately for any data that has *temporal or sequential structure*.

In a sentence, the meaning of a word depends on the words that came before it. In a time-series signal, the value at time t depends on values at $t-1$, $t-2$, and earlier. In speech, each phoneme is conditioned on the phonemes surrounding it. For all such data, the model needs a mechanism to carry information forward across time steps — it needs memory. Feedforward networks have no such mechanism.

A second practical problem is that sequences are variable in length. A standard MLP requires a fixed-size input vector. Sentences have different numbers of words; audio clips have different numbers of frames. Forcing variable-length sequences into a fixed-size input (by padding, truncating, or averaging) destroys the sequential structure that carries meaning.

Recurrent Neural Networks are designed specifically to address both of these problems.

1.2 What is an RNN?

Recurrent Neural Network (RNN): A neural network architecture that processes sequential data one element at a time, maintaining a *hidden state* h_t that is updated at each time step. The hidden state acts as the network's memory, carrying information from earlier time steps forward into later ones.

The key innovation of an RNN is the *recurrence relation*: the hidden state at time step t is computed not just from the current input x_t, but also from the previous hidden state h_{t-1}. This single loop makes the network sensitive to the full history of inputs seen so far.

1.2.1 The Recurrence Relation and Forward Pass

At each time step t, the RNN computes:

$$h_t = \tanh\big(W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h\big)$$

$$\hat{y}_t = W_{hy}\, h_t + b_y$$

where W_{xh} is the input-to-hidden weight matrix, W_{hh} is the hidden-to-hidden (recurrent) weight matrix, W_{hy} is the hidden-to-output weight matrix, and b_h, b_y are bias vectors. The tanh nonlinearity squashes the hidden state into the range $(-1, 1)$.

The critical point is that **the same weight matrices are shared across every time step**. The RNN does not have separate parameters for step 1 versus step 50; it applies the same transformation repeatedly, which is both what allows it to handle variable-length inputs and what makes it parameter-efficient.

Plain language: Think of the hidden state as a running summary of everything the network has seen so far. At each step, you read the new input, update the summary, and produce an output if needed. The update rule stays the same at every step — it is just applied again and again, like reading a book one word at a time and updating your understanding as you go.

1.2.2 The Unrolled RNN

An RNN is typically depicted in its *unrolled* form to make information flow explicit. Rather than showing a single node with a self-loop, you draw one copy of the network for each time step:

- At step $t = 1$: input x_1 enters, combined with initial hidden state h_0 (usually zeros), produces h_1 and optionally $\hat{y}_1$.
- At step $t = 2$: input x_2 enters, combined with h_1, produces h_2 and optionally $\hat{y}_2$.
- This continues for all T time steps.

The unrolled picture makes it clear that the RNN is equivalent to a very deep feedforward network, where depth equals the sequence length and every layer shares the same weights. This equivalence also immediately reveals why training is hard: gradients must flow back through every unrolled time step, just as they flow back through every layer in a deep network.

1.3 Types of RNN Architectures

RNNs are not restricted to one input and one output per sequence. By varying when and how many outputs are produced, four fundamental input/output structures emerge:

Architecture	**Structure**	**Real-World Example**
One-to-One	Single input, single output; no recurrence used	Standard image classification (effectively a plain MLP)
One-to-Many	Single input at step 1, sequence of outputs	Image captioning: one image produces a sequence of words
Many-to-One	Sequence of inputs, single output at the end	Sentiment analysis: a sentence produces one class label
Many-to-Many (equal)	Sequence in, same-length sequence out	Video frame labeling: label each frame of a video
Many-to-Many (unequal)	Sequence in, different-length sequence out	Machine translation: English sentence to French sentence

Common misconception: One-to-one is listed for completeness, but it does not benefit from recurrence. It is the same as a standard feedforward network. The value of RNNs begins with many-to-one or one-to-many structures.

1.4 What RNNs Can Model That MLPs Cannot

- **Sequential context:** The hidden state at step t encodes information from all previous steps $1, \ldots, t-1$. An MLP with a fixed window can only see a fixed number of previous inputs, and even that requires manually engineered input concatenation.
- **Variable-length inputs:** An RNN processes one element at a time and terminates when the sequence ends. It requires no padding or truncation.
- **Parameter sharing across time:** The same W_{xh}, W_{hh}, W_{hy} are reused at every step. An MLP with a fixed large input window would need separate parameters for each position in the window, which does not generalize to different positions.
- **Long-range dependencies (in theory):** Because the hidden state is propagated indefinitely, the network can in principle connect information from step 1 to step 100. In practice, vanilla RNNs struggle to do this, which motivates LSTM.

1.5 Vanishing and Exploding Gradients

Training an RNN uses Backpropagation Through Time (BPTT), which unrolls the network across time steps and applies the chain rule. To update weights at an early time step, gradients must be multiplied together through every intermediate step. If the sequence is T steps long, the gradient involves a product of T Jacobian matrices.

Vanishing gradient: When the largest singular value of the recurrent weight matrix is less than 1, repeated multiplication drives the gradient exponentially toward zero. Early time steps receive almost no gradient signal, so the network cannot learn dependencies spanning many steps.

Exploding gradient: When the largest singular value is greater than 1, repeated multiplication drives the gradient exponentially toward infinity, destabilizing training.

Both problems are present in deep feedforward networks, but they are *far worse* in RNNs because the same weight matrix W_{hh} is multiplied at every step. Even a moderately long sequence of 50 time steps involves 50 multiplications of the same matrix. The vanishing gradient is the more practically damaging of the two, because it produces no error signal at all — the network simply stops learning long-range structure.

Consequences of vanishing gradients in RNNs:

- The model develops *short-term memory*: it can use information from the last few time steps but forgets information from earlier in the sequence.
- Long-range dependencies (e.g., subject-verb agreement across a long clause) become impossible to learn reliably.

Gradient clipping is the standard fix for exploding gradients: if the norm of the gradient exceeds a threshold τ, scale the gradient down so its norm equals τ. This prevents instability without changing the direction of the gradient. Gradient clipping does *not* fix the vanishing gradient problem.

Plain language: Imagine sending a whisper down a long line of people. Each person passes the message on, but introduces a little distortion. By the time it reaches the end of the line, the original message is unrecognizable. Vanishing gradients are the same thing in reverse — the error signal at the end of the sequence barely reaches the beginning, so the beginning of the sequence is never corrected.

1.6 Limitations of Vanilla RNN

- **Poor long-term memory:** Due to vanishing gradients, practical learning horizon is typically only a few dozen time steps.
- **Training instability:** Exploding gradients require careful tuning and gradient clipping.
- **No explicit mechanism to forget:** The hidden state is updated at every step, so old information is continuously overwritten without any learned control over what to keep.
- **Single hidden state:** Information for both short-term processing and long-range context must be packed into the same vector, creating a bottleneck.

1.7 Comparison: RNN vs MLP vs CNN

Property	**MLP**	**CNN**	**RNN**
Primary data type	Fixed-size vectors	Grid-structured (images, signals)	Sequential / temporal data
Handles sequences	No (requires fixed input)	Partially (1D conv over local windows)	Yes (natively, variable length)
Parameter sharing	None across positions	Within each filter (spatial)	Across all time steps
Memory of past inputs	None	Local receptive field only	Carried in hidden state
Typical use case	Tabular data, classification	Image recognition, feature extraction	NLP, time series, speech
Gradient flow difficulty	Moderate (depth)	Moderate (depth)	High (depth $\times$ sequence length)

2 Long Short-Term Memory (LSTM)

2.1 Why LSTM Was Designed

Vanilla RNNs fail in practice on any task requiring the model to remember information more than a few dozen time steps back. The mathematical cause is the vanishing gradient: the recurrent weight matrix W_{hh} is applied at every step, and unless its singular values are carefully maintained near 1 throughout training, gradients either shrink to zero or explode. Hochreiter and Schmidhuber (1997) designed the LSTM specifically to address this: they introduced an architecture in which long-range information can flow through time with *minimal multiplicative interference*, while the network learns to control explicitly what information to keep, discard, or output.

2.2 The Core Idea: Two States

An LSTM maintains two vectors at each time step:

- **Cell state** C_t: a long-range memory vector. Information is added to or removed from C_t through learned gating, but the cell state itself is updated additively, not through repeated matrix multiplication. This is the key to preserving gradients over long sequences.
- **Hidden state** h_t: a filtered version of C_t, used as the output of the LSTM at the current step and passed to the next step's gates. It represents the short-term, "working memory" of the network.

Plain language: Think of C_t as a long conveyor belt running through the sequence. Information is placed on the belt (input gate), removed from the belt (forget gate), or read off the belt (output gate) at controlled checkpoints. The belt itself moves without distortion. h_t is what the worker at each checkpoint actually uses — a focused read of what is currently on the belt.

2.3 The Four Gates

The LSTM controls information flow through four learned gating mechanisms. All gates are computed from the current input x_t and the previous hidden state h_{t-1}.

2.3.1 Forget Gate

$$f_t = \sigma\big(W_f \cdot [h_{t-1},\, x_t] + b_f\big)$$

The forget gate produces a value in $(0, 1)$ for each element of the cell state. A value near 1 means "keep this element"; a value near 0 means "erase this element". The sigmoid activation σ is used because it naturally produces values in $(0, 1)$, making it suitable as a soft binary switch.

2.3.2 Input Gate and Candidate Gate

$$i_t = \sigma\big(W_i \cdot [h_{t-1},\, x_t] + b_i\big)$$
$$\tilde{C}_t = \tanh\big(W_C \cdot [h_{t-1},\, x_t] + b_C\big)$$

The input gate i_t decides *how much* of the new candidate information to write into the cell state. The candidate gate $\tilde{C}_t$ computes *what* new information could be written (values in $(-1, 1)$ via tanh). Together, they determine the net addition to C_t.

2.3.3 Cell State Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

where $\odot$ denotes element-wise multiplication. The cell state is updated by first forgetting some of the old state (element-wise multiply by f_t) and then adding scaled new content (element-wise multiply i_t by $\tilde{C}_t$). This update is **additive**, not multiplicative through a weight matrix, which is what allows gradients to flow back through many steps without vanishing.

2.3.4 Output Gate and Hidden State

$$o_t = \sigma\big(W_o \cdot [h_{t-1},\, x_t] + b_o\big)$$
$$h_t = o_t \odot \tanh(C_t)$$

The output gate determines which parts of the cell state to expose as the hidden state. The cell state is passed through tanh to squash it to $(-1, 1)$, then element-wise multiplied by o_t to produce h_t.

Common misconception: The four gates are not independent switches that learn completely separate things. They are all computed from the same inputs $[h_{t-1}, x_t]$ with different weight matrices. They interact through the cell state update. Thinking of them as "the network decides what to forget, what to write, what to output" is a useful intuition, but the actual learned behavior is more distributed and harder to interpret.

2.4 Why LSTMs Solve the Vanishing Gradient Problem

In a vanilla RNN, the gradient of the loss with respect to the hidden state at step t is multiplied by W_{hh}^T at every step going backward. This repeated matrix multiplication is the source of vanishing gradients.

In an LSTM, the gradient flowing back through the cell state involves the forget gate values f_t — element-wise multiplications, not full matrix multiplications. If the forget gate learns to output values close to 1 for important information, the gradient for those elements flows back unattenuated for many steps. The gradient is never forced to pass through W_{hh}^T repeatedly; it takes the "highway" through C_t instead.

Plain language: In a vanilla RNN, the gradient must squeeze through the same weight matrix at every step going backward — like walking through a series of identical narrow doors. In an LSTM, the gradient can travel through the cell state, which is updated additively. The doors are replaced by an open corridor, and the forget gate decides whether to partially close sections of it. The gradient can travel long distances with minimal loss.

2.5 Advantages of LSTM Over Vanilla RNN

- Captures long-range dependencies reliably (hundreds of steps in practice).
- Explicit gating gives the model fine-grained control over what to remember and what to forget.
- Significantly more stable to train — gradient clipping is less critical.
- State of the art on many sequence tasks before the Transformer era.

2.6 Disadvantages of LSTM

- **Computational cost:** Four gate computations per step instead of one, each involving a matrix multiplication. Inference and training are roughly four times slower than a vanilla RNN of the same hidden size.
- **More parameters:** Four weight matrices and four bias vectors compared to one

each in a vanilla RNN. Risk of overfitting on small datasets.

- **Sequential bottleneck:** Each step depends on the previous hidden state, so the computation cannot be parallelized across time steps. This is the main reason Transformers have replaced LSTMs in many applications.
- **Harder to interpret:** The four-gate mechanism is powerful but opaque; understanding what the network has stored in C_t requires careful analysis.

2.7 When to Use LSTM vs Vanilla RNN

Use a vanilla RNN only for short sequences (up to roughly 20–30 steps) where long-range dependencies are not important and speed or simplicity matters. For nearly all practical sequence modeling tasks — NLP, time series, speech — default to LSTM. If parallelism and very long sequences are critical, consider Transformer architectures instead.

2.8 Comparison: Vanilla RNN vs LSTM

Property	**Vanilla RNN**	**LSTM**
Memory mechanism	Single hidden state h_t	Cell state C_t + hidden state h_t
Gating	None	Forget, input, candidate, output gates
Gradient flow	Degrades rapidly over long sequences	Preserved via additive cell state update
Long-range dependencies	Poor (vanishing gradient)	Good (highway through cell state)
Computational cost	Low	≈4× higher per step
Parameters	Few	Many (four weight matrices)
Typical use case	Very short sequences, simple tasks	NLP, time series, speech, most sequence tasks

3 Bidirectional LSTMs

3.1 The Concept of Bidirectionality

A standard (unidirectional) LSTM processes a sequence from left to right — from x_1 to x_T. At each position t, the hidden state h_t encodes information from the past $(x_1, \ldots, x_t)$ but nothing about the future $(x_{t+1}, \ldots, x_T)$. For many tasks, future context is just as informative as past context.

Consider the sentence: "The bank can guarantee deposits will eventually cover future tuition costs." The word "bank" is ambiguous until you read further. A unidirectional LSTM processing left-to-right cannot resolve this ambiguity at the moment it processes "bank". A bidirectional model can, because it also has access to what comes after.

Bidirectional LSTM (BiLSTM): An architecture consisting of two separate LSTM layers processing the same sequence in opposite directions simultaneously — one forward (left to right) and one backward (right to left). The outputs are concatenated

at each time step to form a representation that incorporates both past and future context.

3.2 Architecture

For a sequence $x_1, x_2, \ldots, x_T$:

- The **forward LSTM** processes $x_1 \to x_2 \to \cdots \to x_T$, producing forward hidden states $\overrightarrow{h}_1, \overrightarrow{h}_2, \ldots, \overrightarrow{h}_T$.
- The **backward LSTM** processes $x_T \to x_{T-1} \to \cdots \to x_1$, producing backward hidden states $\overleftarrow{h}_T, \overleftarrow{h}_{T-1}, \ldots, \overleftarrow{h}_1$.
- At each time step t, the combined representation is:

$$\tilde{h}_t = [\overrightarrow{h}_t ; \overleftarrow{h}_t]$$

where $[;]$ denotes concatenation. If each LSTM has hidden size d, the combined output has dimension $2d$.

Plain language: The forward LSTM reads the sentence normally. The backward LSTM reads it in reverse. At each word position, you take what the forward reader remembers (everything before this word) and what the backward reader remembers (everything after this word), and stick them together. The model at each position effectively sees the full sentence context from both sides.

3.3 Why Bidirectionality Helps

- **Full context at every position:** Every token representation $\tilde{h}_t$ is conditioned on the entire input sequence, not just the prefix up to t.
- **Better disambiguation:** Ambiguous words, pronouns, and named entities that depend on surrounding context are represented more accurately.
- **Richer feature extraction:** The concatenated representation carries twice the information and often leads to significantly better downstream performance on tasks like named entity recognition, sentiment analysis, and question answering.

3.4 When Bidirectionality is Appropriate — and When It Is Not

Critical constraint: Bidirectional LSTMs **cannot** be used for any task requiring real-time or streaming inference. To compute $\overleftarrow{h}_t$ (the backward hidden state at position t), the model must have seen the *entire* future of the sequence, all the way to x_T. This means you cannot generate an output for position t until the whole sequence has been received.

Applications such as speech recognition with low-latency output, real-time machine translation, and online time-series forecasting are therefore incompatible with bidirectional architectures.

Bidirectional LSTMs are appropriate when:

- The entire sequence is available before processing begins (offline tasks).

- Future context genuinely helps — as in sentence classification, named entity recognition, or sentiment analysis.
- Latency is not a constraint.

Unidirectional LSTMs are necessary when:

- Outputs must be produced step-by-step as inputs arrive (streaming, real-time).
- The task is generative — each output token is used as the next input (language model generation, text decoding).

3.5 Advantages and Disadvantages of Bidirectional LSTMs

Advantages:

- Full bidirectional context at every position improves representation quality.
- Particularly effective for classification and tagging tasks.
- Compatible with stacking (deep BiLSTMs) for even richer representations.

Disadvantages:

- Requires the full sequence before any output can be produced.
- Doubles the number of parameters compared to a unidirectional LSTM of the same hidden size.
- Approximately twice the compute and memory per forward pass.
- Incompatible with auto-regressive generation.

3.6 Comparison: Unidirectional LSTM vs Bidirectional LSTM

Property	**Unidirectional LSTM**	**Bidirectional LSTM**
Context seen at step t	Past only $(x_1, \ldots, x_t)$	Past and future $(x_1, \ldots, x_T)$
Output availability	Immediate (after each step)	Only after full sequence processed
Parameter count	d hidden units per direction	$2d$ effective (two LSTM layers)
Streaming/real-time use	Yes	No
Classification / tagging	Good	Better
Language model generation	Yes	No (cannot generate autoregressively)
Typical use cases	Speech synthesis, text generation, forecasting	Sentiment analysis, NER, machine translation encoding

4 Case Study: Sentiment Analysis

4.1 Task Definition

Sentiment Analysis: A natural language processing task that assigns a sentiment label to a piece of text. The label may be binary (positive / negative), ternary (positive / neutral / negative), or a finer-grained score. The input is a variable-length text (word, sentence, paragraph, or document); the output is a class label or probability distribution over labels.

Sentiment analysis is one of the most widely studied NLP tasks and a natural benchmark for sequence models, because the sentiment of a sentence frequently depends on the entire sequence — not just individual words in isolation.

4.2 Why Sentiment Analysis is a Natural Fit for RNN/LSTM

Text is inherently sequential. Words carry meaning through their order and context. Consider the difference between "not bad" and "bad not" — the same words, different order, entirely different sentiment. A bag-of-words model, which ignores order, cannot distinguish them. An RNN/LSTM processes words sequentially and builds a representation that is sensitive to order and context.

Additionally, sentiment depends on long-range dependencies:

- Negation: "I would have enjoyed this product if it had not broken within the first week." The word "not" appears near the end and reverses the polarity of a positive-seeming sentence.
- Contrast: "The acting was brilliant, but the plot was incoherent." Overall sentiment requires reasoning about how two opposing clauses interact.
- Conditional: "This is a great movie if you like three-hour films with no action." The conditional clause modifies the main sentiment.

LSTMs, with their cell state preserving long-range information, can in principle carry the negation signal across many steps to the relevant adjective.

4.3 End-to-End Pipeline

4.3.1 Stage 1: Raw Text

The input is a raw string — a product review, a tweet, a customer comment. No structure is assumed beyond character sequences.

4.3.2 Stage 2: Tokenization

The text is split into tokens (usually words, but sometimes subwords). Punctuation may be separated; case may be lowered. The output is a sequence of tokens $[w_1, w_2, \ldots, w_T]$.

Why it matters: The model operates on discrete symbols, not raw characters. Tokenization converts continuous text into a fixed vocabulary of meaningful units.

4.3.3 Stage 3: Word Embedding

Each token is mapped to a dense vector $x_t \in \mathbb{R}^d$ via an embedding matrix. Pre-trained embeddings (Word2Vec, GloVe, FastText) are commonly used to initialize this layer, so the model begins with semantic relationships already encoded (e.g., "good" and "excellent" map to nearby vectors).

Why it matters: Neural networks require continuous inputs. Embeddings convert discrete tokens into vectors that can be processed by the LSTM and trained end-to-end. Pre-trained

embeddings provide a strong starting point on limited data.

4.3.4 Stage 4: LSTM Layers

The sequence of embedding vectors $[x_1, x_2, \ldots, x_T]$ is fed into one or more LSTM layers. At each step, the LSTM updates its cell state and hidden state. After processing the full sequence, the LSTM has built a hidden state that encodes the sequential context of the entire text.

Why it matters: The LSTM accumulates contextual information across the sequence, capturing word order, negations, and long-range dependencies that earlier layers (embedding alone) cannot represent.

4.3.5 Stage 5: Pooling / Aggregation

The LSTM produces a hidden state at every time step. To produce a single fixed-size vector for classification, one of the following strategies is used:

- **Last hidden state:** Take h_T, the hidden state after the final word. Common for many-to-one tasks.
- **Max pooling:** Take the element-wise maximum across all time steps. Captures the strongest signal at any position.
- **Mean pooling:** Average $h_1, \ldots, h_T$. Smooths across the sequence.

Why it matters: The downstream classifier (dense layer) requires a fixed-size input regardless of sequence length. Pooling is the bridge between variable-length LSTM output and the classifier.

4.3.6 Stage 6: Dense Layer and Softmax Output

The aggregated vector is passed through one or more fully connected layers, then a softmax (for multi-class) or sigmoid (for binary) output layer to produce a probability distribution over sentiment classes. The predicted label is the class with highest probability.

Why it matters: The LSTM output is a rich but high-dimensional representation. The dense layer learns to map this to the small number of target classes, and the softmax normalizes the output into interpretable probabilities.

4.4 How the LSTM Captures Sentiment

- **Negation:** When the LSTM processes "not", the forget and input gates can learn to flag the upcoming adjective for polarity reversal. The cell state carries this negation signal forward to the relevant word.
- **Intensity modifiers:** Words like "very", "extremely", and "barely" modify the strength of the following sentiment word. The LSTM can learn to amplify or attenuate the cell state accordingly.
- **Contrastive conjunctions:** "But" and "however" often signal a sentiment shift. The LSTM can learn to weight the clause following "but" more heavily.
- **Accumulation:** Multiple positive signals across a long review accumulate in the cell state, producing a strong positive sentiment representation even if no single sentence is decisive.

Plain language: The LSTM reads the sentence like a careful reader who keeps a mental note of the overall tone. When it encounters "not", it updates its note: "what comes next should be interpreted in reverse". When it encounters "but", it updates: "what comes next may override what I thought earlier". By the end of the sentence, the mental note captures the overall sentiment.

4.5 Why Bidirectional LSTM is Preferred for Sentiment Analysis

Sentiment analysis is an offline task: the entire review is available before any prediction is needed. This makes it compatible with a bidirectional architecture. The advantage is substantial:

- A word like "disappointing" near the beginning of a review is better represented when the model has also seen how the rest of the review qualifies or reinforces it.
- In "Despite some flaws, the film was genuinely moving and deeply satisfying", the word "flaws" is much better understood when the forward LSTM has seen the concessive structure and the backward LSTM has seen the positive conclusion.
- The final pooled representation benefits from both the forward (left-to-right) and backward (right-to-left) reading, producing a richer and more accurate sentiment embedding.

Exam note: When asked why BiLSTM outperforms unidirectional LSTM for sentiment analysis, the core answer is: sentiment analysis is an offline, whole-sequence task. Every position benefits from full bidirectional context, and there is no streaming constraint that would prohibit it. Future context often carries polarity-modifying information that helps correctly classify sentiment.

4.6 Real-World Applications

- **Product reviews:** E-commerce platforms classify customer reviews to compute star ratings, flag problematic content, and identify common complaints.
- **Social media monitoring:** Companies track brand sentiment on Twitter and Reddit in near-real time to detect crises, measure campaign effectiveness, and respond to customer concerns.
- **Customer feedback analysis:** Call centers and support ticket systems classify incoming messages by sentiment and urgency to prioritize routing.
- **Financial sentiment:** News headlines and analyst reports are classified for market sentiment to support trading strategies.
- **Political analysis:** Public opinion on policy issues is measured from social media and news commentary.

4.7 Limitations of RNN/LSTM-Based Sentiment Analysis

- **Sarcasm and irony:** "Oh great, another delay" is positive in form but negative in intent. Sarcasm requires world knowledge and pragmatic reasoning that LSTMs do not have. Performance degrades sharply on sarcastic text.
- **Complex negation:** "It's not that I didn't enjoy it" involves double negation and

is difficult to parse correctly without deep syntactic understanding.

- **Domain shift:** A model trained on movie reviews will perform poorly on medical feedback because vocabulary, style, and sentiment expressions differ significantly. Transfer requires fine-tuning on in-domain data.
- **Long documents:** Even LSTMs struggle on very long documents (thousands of words) because the pooling step compresses too much into a single vector. Attention mechanisms and Transformers handle this better.
- **Implicit sentiment:** "The battery lasts two hours" implies negative sentiment (for a laptop) without using any explicitly negative word. This requires background knowledge, not just sequential pattern matching.
- **Multilingual and code-mixed text:** Social media text often mixes languages or scripts, which pre-trained monolingual embeddings cannot handle without dedicated multilingual models.